EVIDENCE ATTACHMENT

Apranik: Evidence of the Internet Freedom Infrastructure Gap in Iran

A public-source review of Iran's 2026 Internet shutdowns, the circumvention techniques developed in response, and the ecosystem barriers that separated those techniques from the users who needed them.

2
nationwide Internet shutdowns in 2026

86 days
duration of the second shutdown, enforced by filtering

4+
publicly documented circumvention projects, on three substrates

0
of them built into an established proxy core

> Apranik is a working prototype of an open-source proxy core, developed in the open at https://github.com/we-apranik/apranik under MPL 2.0. This document sets out the problem that motivated it. All sources are public and listed in section 7.

CONTENTS

Executive summary 2

1 Introduction and purpose 2

2 Iran’s censorship environment and the 2026 crisis 3

3 Circumvention innovation during the crisis 5

4 From innovation to deployment: the ecosystem gap 7

5 From evidence to infrastructure: the Apranik response 9

6 Conclusion 12

7 Sources and references 13

Executive summary

Central finding. Iran's 2026 Internet crisis demonstrated both a severe censorship problem and a strong technical response from Iranian developers. When established circumvention methods became ineffective or unreachable, researchers and developers produced or adapted alternative approaches including DNS tunnelling, Google-service relays with domain fronting, and Google Drive transport. The important gap was not the absence of technical ideas. It was the gap between a promising technique and a capability that ordinary users could access through familiar clients, maintain and deploy at scale. Apranik is a modular, open-source proxy platform, reusable infrastructure built to shorten that path.

Four findings:

1. **Iran's censorship can remove the network paths that circumvention depends on.** Two nationwide shutdowns in 2026 reduced international connectivity to a very small fraction of baseline, and the second, enforced by filtering rather than by withdrawing routes, lasted 86 days. Under such conditions, rotating between conventional proxy protocols is not sufficient, because the path to the proxy may itself be unreachable. (Section 2)
2. **Iranian and Iran-focused developers responded with genuine innovation.** At least four distinct publicly documented projects appeared or were adapted during the crisis, spanning three different communication substrates. The constraint on Internet Freedom in Iran is not a shortage of circumvention research. (Section 3)
3. **Technical success and broad user availability are different outcomes.** Each of these techniques required its own client and server components, relay setup or bespoke configuration before an ordinary user could benefit from it, and none is built into existing proxy cores. (Sections 3 and 4)
4. **The gap is structural, and it recurs with every censorship change.** Because the useful lifetime of a technique can be shorter than the time required to implement, integrate, test and distribute it, every additional integration step reduces the number of people a successful technique can reach. (Section 4.5)

What follows. Iran does not lack circumvention research. It lacks reusable infrastructure for turning that research into capabilities ordinary users can reach. Apranik is built to address that gap (Section 5).

1 Introduction and purpose

This document is intentionally narrower than a general history of Internet censorship in Iran, a catalogue of circumvention tools, or a technical tutorial. Its purpose is to substantiate the specific problem that motivates the project.

The central question is not whether Iranian and Iran-focused developers can invent circumvention techniques. The public record demonstrates that they can. The question is what happens after a promising technique is discovered:

> *How quickly can research become a secure, maintainable capability that ordinary users can access through the clients and applications they already use?*

The evidence reviewed here suggests that this transition can become a significant bottleneck during rapidly changing censorship events.

Apranik is a modular proxy platform infrastructure: an open-source proxy core in which a new circumvention technique is added as one self-contained module.

Relationship to existing platforms. Existing platforms including Xray-core, sing-box and Mihomo are valuable components of the Internet Freedom ecosystem, optimised for general-purpose proxy functionality, security, compatibility, stability and long-term maintenance across broad user communities. The narrower question asked here is whether the ecosystem also provides a platform whose architecture and community priorities are specifically optimised for rapid, community-driven adaptation to new censorship techniques. [8] [9] [10] This attachment therefore examines the gap between innovation and deployment, rather than judging the quality or success of existing projects.

2 Iran’s censorship environment and the 2026 crisis

2.1 The relevant characteristic: censorship can change the available network paths

Iran’s censorship environment extends beyond conventional website blocking. Public research has documented combinations of DNS interference, protocol filtering, throttling, deep packet inspection, selective reachability and whitelisting. Earlier research also documented severe throttling and selective protocol blocking in conditions where the Internet remained partially reachable. [7]

One finding from that research is directly relevant to everything that follows. On ordinary days, outside any shutdown, the filtering infrastructure has been documented as operating a strict protocol whitelist, in which DNS, HTTP and HTTPS are forwarded while other traffic is silently dropped, and as inspecting hostnames, injecting resets for banned content, and throttling or blocking HTTPS, SSH and VPN protocols. [7] A network shaped this way does not merely make circumvention harder. It determines which substrates remain available at all, and it predicts that developers under pressure will be pushed toward DNS and HTTP. Section 3 shows that this is what happened. A shutdown is a different and harsher state, described next.

This distinction matters for circumvention because the useful communication path depends on the censorship state.

- A method that disguises proxy traffic inside ordinary HTTPS can be useful when general Internet connectivity exists but specific destinations or fingerprints are blocked.
- A DNS-based method can become relevant when DNS remains reachable while ordinary application connections are restricted.
- A cloud-service relay or domain-fronting technique can become relevant when particular globally reachable services remain accessible.
- During a near-total international shutdown, the problem changes again: **the remaining communication channels themselves become the substrate that circumvention must exploit.**

2.2 The January 2026 nationwide shutdown

On 8 January 2026, Iran entered a nationwide Internet shutdown amid nationwide protests. Independent measurement organisations documented a dramatic collapse in connectivity. Cloudflare observed a near-total loss of traffic. [1] IODA likewise recorded severe nationwide loss of reachability, with active-probing responsiveness falling to about 3%, and called it the most severe shutdown in Iran's history. [2] Press reports quoting NetBlocks, Cloudflare, Kentik and Miaan Group described a near-total disconnection from the outside world. [3] Academic research subsequently reconstructed the event from multiple public datasets and analysed both the shutdown and the circumvention response. [4]

The shutdown came with a violent crackdown on the protesters. Amnesty International stated that the authorities "deliberately blocked internet access inside Iran to hide the true extent of the grave human rights violations", and that the blackout prevented abuses from being documented. [20] Human Rights Watch likewise reported that the shutdown severely hampered efforts to corroborate unlawful killings and other violations. [21]

The important characteristic is not that the Internet became slow: normal international connectivity was reduced to a very small fraction of baseline.

2.3 Partial restoration did not mean a return to normal

The January shutdown was followed by partial and uneven restoration rather than an immediate return to normal connectivity. Internet Society Pulse records partial restoration and continuing restrictions, while subsequent measurements documented instability and filtering. [5]

The operating environment moved through several distinct states, from heavy filtering and whitelisting to near-total disconnection and partial restoration, and a circumvention mechanism that works in one state may fail in another. This makes **adaptation speed** a practical part of Internet Freedom resilience.

2.4 The second 2026 nationwide shutdown

A second nationwide shutdown began on 28 February 2026 as military strikes escalated. Cloudflare observed traffic from Iran falling to well under 1% of previous levels, with only small amounts of web and DNS traffic still leaving the country. [6] It saw no significant shift in announced IP address space at the onset (IPv4 steady, IPv6 volatile throughout), which suggests the shutdown was enforced through filtering rather than route withdrawal, and cites reports of “whitelists” and “white SIM cards” restricting access to approved sites and users. [6]

A multi-perspective academic study of the second shutdown (BGP data, active probing and passive scans) puts its duration at 86 days, from 1 March to 26 May 2026. [11]

These findings reinforce that the 2026 events were not simply periods of website filtering: the reachable network itself could become highly selective.

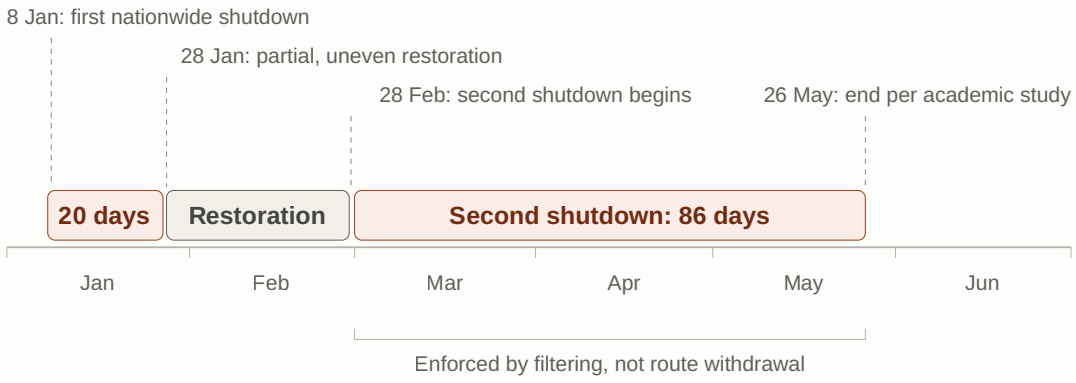


Figure 1. Iran's two nationwide Internet shutdowns in 2026. Dates are from Internet Society Pulse [5] (8 and 28 January), Cloudflare [6] (28 February) and an academic study [11] (end date and 86-day duration). The second bar follows the study's window, which starts a day after Cloudflare's reported onset; Cloudflare indicates the shutdown was enforced through filtering rather than route withdrawal [6].

3 Circumvention innovation during the crisis

The 2026 response included several technically distinct approaches. Some techniques had prior art and were adapted or newly implemented for Iranian conditions; others became prominent during the crisis. The important observation is that developers were experimenting with **different communication mechanisms** as the available network paths changed.

3.1 DNS-based circumvention: MasterDNSVPN

MasterDNSVPN is a DNS-tunnelling proxy implementation that carries proxy traffic through DNS queries and responses. Its documentation describes reliability mechanisms, resolver selection, failover, packet-loss handling and other measures intended to make DNS tunnelling more usable on constrained links. [12] [13]

MasterDNSVPN should not be understood as a high-performance replacement for conventional proxy transports. DNS has severe throughput, latency, resolver, rate-limit and filtering constraints. Its value in the Iranian context is different: it represents an alternative communication path that can remain usable under conditions in which ordinary proxy connections are unavailable. RaazNet documents MasterDNSVPN, GooseRelayVPN and Skirk as prominent Iranian-oriented approaches developed around the 2026 crisis. [14]

3.2 Cloud-service relays: GooseRelayVPN and MasterHTTPRelayVPN

Two projects converged on the same idea from slightly different directions. GooseRelayVPN exposes a local SOCKS5 interface and routes raw TCP through a Google Apps Script relay toward a user-controlled exit server, documenting separate client and server components and the use of an externally reachable Google service as the intermediary. [15] MasterHTTPRelayVPN uses a related model built on Google Apps Script with HTTP/SOCKS5 relay behaviour, adding domain-fronting and local interception components. [16]

Neither provides ordinary high-speed Internet access. What both change is the communication path, so that the censored network need not permit a direct connection to the exit server. Both also carry operational constraints: a relay and exit must be configured, the intermediary service must remain usable, and performance depends on the resulting path.

3.3 Google Drive transport: Skirk

Skirk uses Google Drive as an intermediary mailbox for encrypted traffic and exposes local proxy interfaces to applications. Its design depends on object upload, visibility, listing, download and service/API behaviour rather than a conventional low-latency interactive network path. [17]

Skirk therefore has substantial performance and reliability limitations. Its significance is not high-speed access; it is the ability to use a communication substrate that may remain available when ordinary network paths do not.

3.4 Cross-project conclusion

The projects differ technically, but together they demonstrate a consistent pattern.

Each shipped as its own program, with its own client, server or relay components, and none is part of Xray-core, sing-box or Mihomo.

Two things are worth stating. Most of these were specialised or emergency mechanisms rather than high-performance general transports; and their effectiveness depended on the exact censorship state, changing as filtering adapted.

The lesson is therefore not that emergency techniques should replace mature proxy protocols. It is that **a mature Internet Freedom ecosystem needs a low-friction way to incorporate unusual techniques when the ordinary stack stops working.**

4 From innovation to deployment: the ecosystem gap

4.1 Technical success is only the beginning

The path from a research result to ordinary-user access is longer than producing a proof of concept. The chain runs from research to a proof of concept, then through production implementation, proxy-platform integration, client integration and distribution before reaching user adoption. A working experiment may still need production hardening, configuration management, client support, server infrastructure, testing and release engineering before it is practical for a non-technical user.

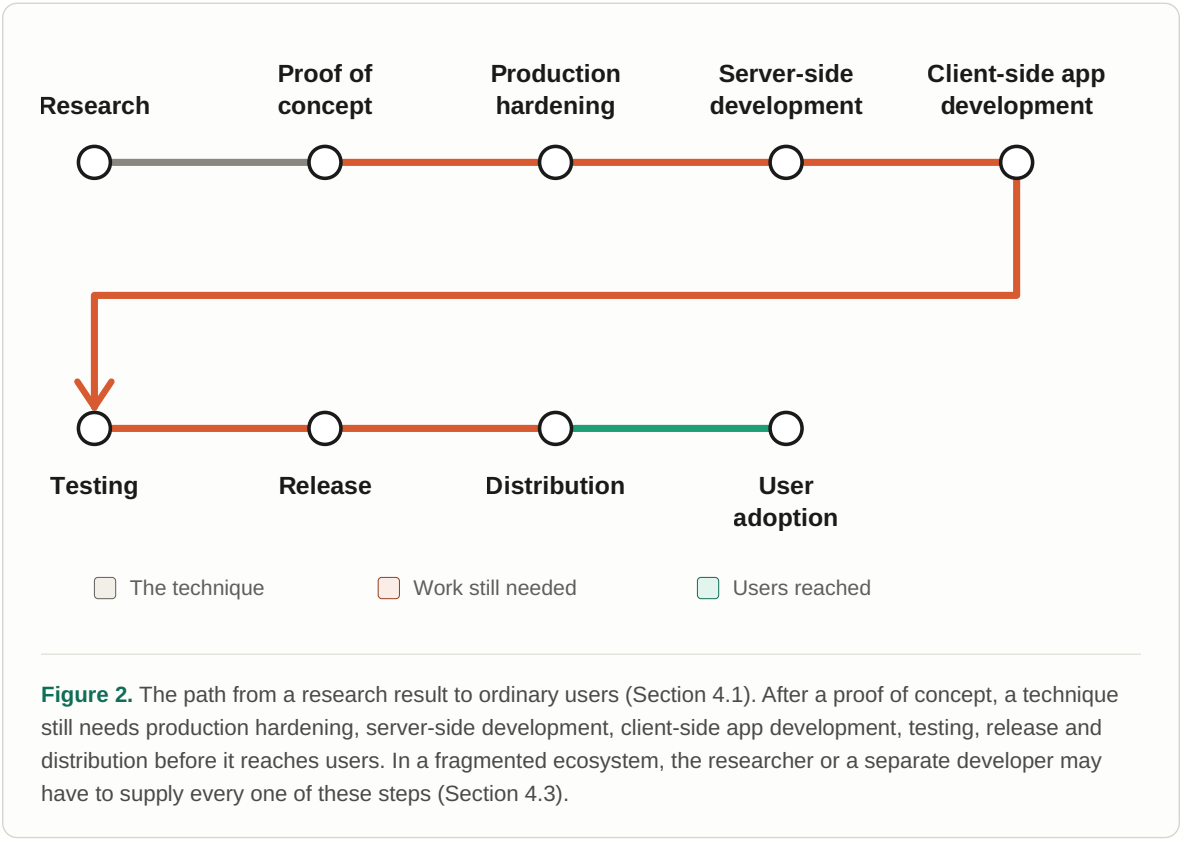


Figure 2. The path from a research result to ordinary users (Section 4.1). After a proof of concept, a technique still needs production hardening, server-side development, client-side app development, testing, release and distribution before it reaches users. In a fragmented ecosystem, the researcher or a separate developer may have to supply every one of these steps (Section 4.3).

4.2 The client ecosystem is the scaling mechanism

Running a proxy core directly, with a configuration file on the command line, is work for technically skilled people. Almost everyone else uses a client app or a server panel built on top of a core, such as Hiddify or PattNG [18] [19], and even skilled users choose them because they are easier. A technique built into the core therefore reaches every app and panel that uses that core, with no extra step for the user. A technique outside the core reaches only the people who can install and run a separate program themselves, or it must be added to each app and panel separately. How many people benefit from a successful technique depends on whether it lives in a shared core.

4.3 Repeated infrastructure work

A researcher working on a new censorship technique should ideally be able to concentrate on the technique itself. In a fragmented ecosystem, however, the same researcher or a separate developer may also need to provide client/server plumbing, configuration handling, connection management, routing, security and session handling, packaging, platform-specific integration, and ongoing maintenance.

This pattern holds across all four projects: every one of them shipped or required infrastructure surrounding the technique itself. [12] [15] [16] [17]

The issue is not that standalone projects are inherently undesirable. They are often the fastest way to prove a new idea. The problem arises when every successful technique must also reconstruct the surrounding production infrastructure before it can reach a large user base.

4.4 Why existing platforms do not automatically remove the gap

Established platforms can technically implement many new features. The question is whether every new censorship technique fits naturally into their current abstractions, release priorities and contributor workflow.

This does not show that existing platforms are inadequate in general. It shows that **rapid adaptation to new censorship techniques may require work outside the normal development path of a mature general-purpose platform.**

4.5 Timing and obsolescence

Section 2 showed Iran's network moving through several states within months: heavy filtering and whitelisting, near-total shutdown, partial restoration, and a second shutdown. A technique that works in one state may fail in the next, so its useful life can be short. Each added step between a working technique and its users (implementation, integration, a client release, deployment) adds delay that competes with that useful life. The four projects above each reached users as a separate program, so every one of them paid this cost on its own.

4.6 The actual infrastructure gap

The evidence supports a precise conclusion. **The bottleneck is in the repeated integration work, not in the supply of research and not in the supply of proxy protocols.** What is missing is a sufficiently reusable, community-oriented platform that can turn a successful research result into a production-capable capability and deliver it through the client ecosystem without repeating the surrounding engineering work each time. That is the problem Apranik is built to address.

5 From evidence to infrastructure: the Apranik response

5.1 What the evidence requires

The evidence above implies a small set of practical requirements for a rapid-response Internet Freedom platform.

Requirement	What it means in practice
Modularity and separation of concerns	New protocols, transports or communication mechanisms should be implementable without invasive changes elsewhere. Protocol logic should not need to know the details of every communication mechanism; new mechanisms should plug into stable platform interfaces.
Reusable infrastructure	Researchers should be able to reuse connection management, routing, configuration, security and session infrastructure, and client/server plumbing rather than rebuilding them for each technique.
A path from experimentation to production	A successful research implementation should have a clear route through testing, security review, packaging, release and client integration.
Community maintainability	The platform should be understandable and extensible by contributors from the affected community, with documentation, testing, transparent review and predictable releases.
Client interoperability	The platform must be capable of serving the applications that Iranian users already use, so that a capability implemented once can potentially reach many users without every client independently reimplementing it.
General-purpose utility	The platform must also support normal, reliable circumvention use. Emergency techniques have value only if there is a production ecosystem capable of delivering them when needed.

Table 1. Platform requirements implied by the evidence in Sections 2 to 4.

5.2 The Apranik platform

Project status. Apranik is a working prototype of an open-source core, developed in the open at https://github.com/we-apranik/apranik under MPL 2.0. It is not complete. It does not yet carry traffic over the non-stream substrates of section 3 (DNS tunnels, cloud relays, mailboxes); building the first such modules (a DNS tunnel, then a cloud-service relay) on the existing contract, and publishing what each cost, is planned.

The architecture, as built, has four layers, each a set of modules on one small contract. Client applications sit above them and run the `apranik` binary (or link the Go library) with a config file:

- **Carriers**: two kinds only, a **stream** and a **datagram**, handed between layers. Substrates with no server initiative, such as DNS tunnels and mailbox relays, are meant to present a stream to the layers above.
- **Base transports** (TCP, UDP) create connections; future methods go here.
- **Overlay transports** (TLS with browser fingerprints, WebSocket, HTTPUpgrade, TLS fragmentation) wrap an existing connection.
- **Proxy protocols** (VLESS, Trojan, SOCKS5, HTTP, and others to come) speak proxy semantics over a carrier they are handed and never build a transport.

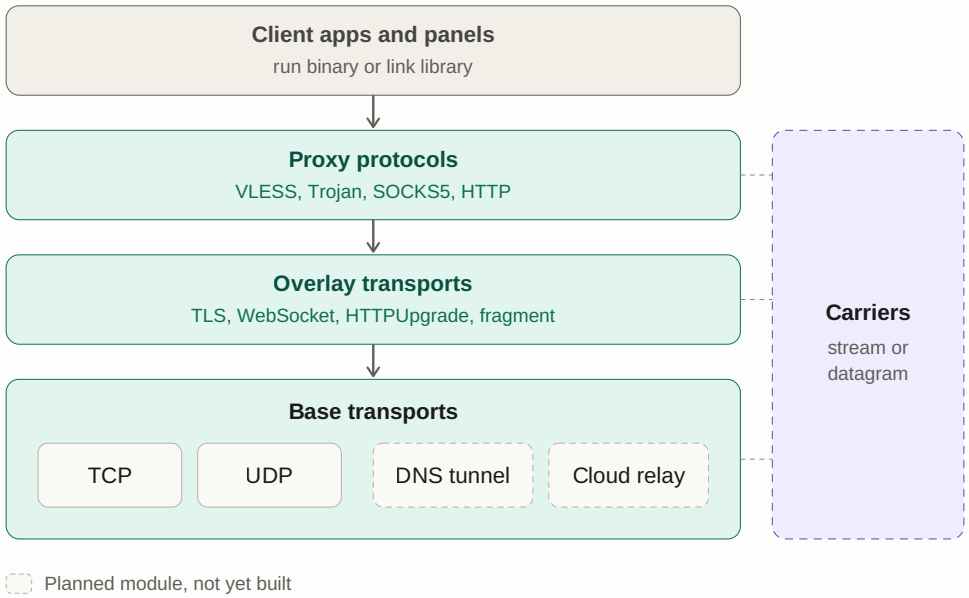


Figure 3. Apranik's layered architecture as built (Section 5.2). Each layer is a set of modules on one small contract, and carriers (a stream or a datagram) are handed between layers. Dashed outlines mark planned modules that do not yet exist.

The architectural property is that a protocol implementation does not need to know whether its underlying communication path is TCP, a WebSocket over TLS, a DNS-based path or a cloud relay, and that a new communication mechanism does not require changes to every protocol that may use it. Adding a module is a new folder plus one line in a registry.

5.3 How the design answers the documented barriers

The barriers in sections 3 and 4 were a separate client and server for each technique, substrates that are not live connections, and slow, repeated integration. In Apranik a technique is a module on one contract that reuses the existing protocols, overlays, routing, DNS, users and statistics, ships inside the core, and needs no edits to other modules. This is checkable in the code today: each transport overlay is referenced from one place outside its own folder, the registry line that names it. The first non-stream modules, a DNS tunnel and a cloud-service relay, are planned.

5.4 From researcher to ordinary user

A researcher who discovers a new technique writes an Apranik-compatible module; it is tested and reviewed, ships in an Apranik release, and reaches ordinary users through the apps and panels that run Apranik. A new technique does not automatically become fast, reliable or universally useful this way, but it gets a shared technical path to users, instead of requiring its author to recreate the proxy runtime, routing, configuration, client integration and server infrastructure.

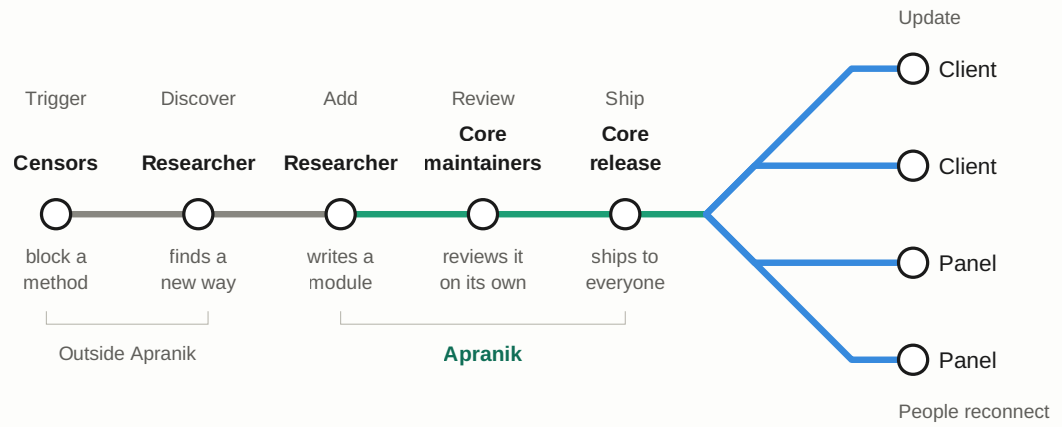


Figure 4. From a censorship change to people reconnecting (Section 5.4). Apranik covers the add, review and ship steps; the release then reaches users through the clients and panels that run Apranik.

5.5 Why a new platform rather than only extending or forking existing projects?

The case for a new platform rests on two considerations.

Architecture. In our reading, the established cores are structured so that adding a new circumvention method is hard to do easily, especially one that departs from an ordinary connection. We believe this is why none of the four techniques in section 3 was integrated into an existing core. A fork can provide a new maintenance home, but it inherits the same structure. That is why Apranik is modular: rapid experimentation and integration are requirements that should shape the architecture from the beginning, so that a new technique is one module.

Community priorities. Existing projects serve broad international communities and must balance many priorities, so a method whose main benefit is for people in Iran is, understandably, not their first claim on limited time and energy. What is needed is a platform in which responding to censorship is a primary development concern, and in which developers with direct knowledge of the environment can take part directly in the roadmap and implementation.

5.6 Iran-first, globally reusable

The primary motivation for Apranik is Iran. The team has direct experience with the Iranian environment, and the 2026 crisis provides a concrete demonstration of the infrastructure gap documented in this attachment.

At the same time, the software is open source and designed as reusable infrastructure. Developers and communities elsewhere can adopt, modify and extend it according to their own censorship conditions.

6 Conclusion

Iran's 2026 crisis showed both a severe censorship problem and a capable technical response. The deeper problem appears after a technique is demonstrated. DNS tunnelling, Google-service relays and Google Drive transport each worked well enough to matter, yet each reached users as its own program with its own client, server or relay setup, and none became part of a widely used proxy core. Setting one up meant installing and configuring something separate, so in practice only technically capable people used them; ordinary people, who rely on apps built on the established cores, could not. [12][15][16][17]

The evidence therefore supports a narrower and more useful conclusion than “Iran lacks circumvention tools”:

> *Iran has circumvention research and a capable technical community, but it lacks a sufficiently reusable, community-driven proxy platform that can rapidly turn new circumvention research into production-capable capabilities and distribute them through the client ecosystem.*

Apranik is built to address that gap. The objective is not to guarantee that every future method will work, nor to replace existing proxy ecosystems. It is to reduce the repeated infrastructure and integration work that currently separates a promising circumvention technique from the users who need it.

7 Sources and references

All sources below are publicly accessible. They were accessed in August 2026 and checked again in October 2026. Where sources differ on a point of detail, the difference is noted in the body text.

[1] Cloudflare, Iran Internet shutdown reporting and measurements, January 2026. https://blog.cloudflare.com/iran-protests-internet-shutdown/

[2] IODA / Georgia Tech Internet Intelligence Lab, A comparative look at Internet shutdowns in Iran. https://ioda.inetintel.cc.gatech.edu/reports/a-comparative-look-at-internet-shutdowns-in-iran-2019-2022-2026-and-2026/

[3] TechCrunch, Internet collapses in Iran amid protests, 8 January 2026. https://techcrunch.com/2026/01/08/internet-collapses-in-iran-amid-protests-over-economic-crisis

[4] Academic research, Iran’s January 2026 Internet shutdown, censorship and circumvention. https://arxiv.org/abs/2603.28753

[5] Internet Society Pulse, Blackout in Iran. https://pulse.internetsociety.org/en/shutdowns/blackout-in-iran

[6] Cloudflare, Q1 2026 Internet disruption review, including Iran. https://blog.cloudflare.com/q1-2026-internet-disruption-summary/

[7] Academic research, Iran’s censorship architecture and filtering model. https://arxiv.org/abs/2507.14183

[8] XTLS, Xray-core. https://github.com/XTLS/Xray-core

[9] SagerNet, sing-box. https://github.com/SagerNet/sing-box

[10] MetaCubeX, Mihomo. https://github.com/MetaCubeX/mihomo

[11] Academic study, a multi-perspective study of the 2026 Iranian Internet shutdown using BGP, active probing and Censys data. https://arxiv.org/abs/2605.00187

[12] MasterDNSVPN. https://github.com/masterking32/MasterDnsVPN

[13] MasterDNSVPN releases. https://github.com/masterking32/MasterDnsVPN/releases

[14] RaazNet, review of Iranian alternative-access efforts including GooseRelayVPN, Skirk and MasterDNSVPN. https://raaznet.com/fa/blog/gooserelay-skirk-masterdns-iran-creative-method-vpn

[15] GooseRelayVPN. https://github.com/Kianmhz/GooseRelayVPN

[16] MasterHTTPRelayVPN. https://github.com/masterking32/MasterHttpRelayVPN

[17] Skirk. https://github.com/ShahabSL/Skirk

[18] Hiddify App. https://github.com/hiddify/hiddify-app

[19] Patterniha, PattNG. https://github.com/patterniha/PattNG

[20] Amnesty International, Iran: Internet shutdown hides violations in escalating deadly crackdown on protesters, 9 January 2026. https://www.amnesty.org/en/latest/news/2026/01/internet-shutdown-in-iran-hides-violations-in-escalating-protests/

[21] Human Rights Watch, Iran’s Internet Blackout Concealing Atrocities, 12 January 2026. https://www.hrw.org/news/2026/01/12/irans-internet-blackout-concealing-atrocities